MANUAL

build & bring-up

SMART HOME HUB · RASPBERRY PI 4 · ARDUINO DUE · REV 9916F43 · 3 AUGUST 2026

Step-by-step assembly and commissioning manual for the smart home hub: wiring the sensors, provisioning the WiFi devices, bringing the stack up on the MacBook, then migrating it to the Raspberry Pi 4 for 24/7 use.

Read `CLAUDE.md` first for the architecture. This manual is the *doing* half; it assumes the design decisions there are settled.

> ⚠ **Read this before you touch the lighting**
>
> **The code does not speak Shelly yet.** `CLAUDE.md` describes Shelly Multicolor Bulb E27 Gen3 zones, but that decision was documented ahead of the implementation. The backend, dashboard, database, and tests still target **WLED** (`app/wled.py` , device type `wled_zone` , `WLED_*_IP` env vars). The bulbs will provision onto WiFi and answer `curl` fine, but the hub cannot drive them until the migration in §5.4 is done.
>
> Everything else — sensors, both plugs, scenes, planner, health — is complete and tested. Plan the lighting zone as the last thing you commission.

CONTENTS

- 0. Plan and parts
- 1. Stage 0 — dry run on the Mac, no hardware
- 2. Stage 1 — wired lane: Arduino Due + sensors
- 3. Stage 2 — connect the Due to the Mac
- 4. Stage 3 — WiFi lane: myStrom plugs
- 5. Stage 4 — WiFi lane: Shelly bulbs
- 6. Stage 5 — full verification on the Mac
- 7. Stage 6 — the Raspberry Pi 4
- 8. Day 2 — making changes
- 9. Troubleshooting
- Appendix A — environment variables
- Appendix B — serial protocol card
- Appendix C — useful one-liners

0. Plan and parts

The order to build in

Each stage is verifiable on its own. Do not skip ahead — the point of the ordering is that when something breaks you already know the layer below it works.

STAGE	WHAT	VERIFIED BY
0	Stack runs on the Mac with `MOCK_HARDWARE=1`	Dashboard renders, charts move
1	Sensors wired, firmware flashed	Arduino serial monitor shows `TEMP:` etc.
2	Due talks to the Mac	`Serial connected` in the log, live sensor widgets
3	myStrom plugs on WiFi	`curl .../report`, dashboard toggle switches a lamp
4	Shelly bulbs on WiFi	`curl .../rpc/RGBCCT.GetStatus` (**hub control blocked, see §5.4**)
5	Everything together on the Mac	The checklist in §6
6	Same thing on the Pi, running 24/7	systemd service survives a reboot

Parts checklist

Wired lane

- ☐ Arduino Due (3.3 V logic — **pins are not 5 V tolerant**)
- ☐ USB micro-B cable (for the **programming port**, the socket nearer the DC jack)
- ☐ Breadboard + jumper wires (male-male; male-female if your sensors have headers)
- ☐ BME280 breakout — temperature + humidity (I2C, 0x76 or 0x77)
- ☐ BH1750 breakout — ambient light (I2C, 0x23)
- ☐ SCD40 breakout — CO2 (I2C, 0x62)
- ☐ HC-SR501 PIR — motion (digital)

All I2C boards must be **breakouts with onboard regulation** (Adafruit-style), never bare sensor chips.

Wireless lane

- ☐ 2 × myStrom WiFi Switch (Swiss Type J)
- ☐ 2 × Shelly Multicolor Bulb E27 Gen3
- ☐ 2 × E27 fixture (the fixture only ever supplies mains power — all control is inside the bulb, so its wall switch has to stay **ON** permanently)

Host

- ☐ Raspberry Pi 4, 4 GB
- ☐ microSD card, 32 GB, **high-endurance** (SanDisk Max Endurance, Samsung Pro Endurance or similar) — OS, database and backups all live on it. Buy for the endurance rating, not the size; §7.1 explains why
- ☐ USB-C power supply, official or equivalent (5 V / 3 A)
- ☐ The old MacBook, for stage 0–5 and as a fallback host

Also needed: admin access to the router (for static DHCP reservations), the myStrom app and Shelly Smart Control app (initial WiFi provisioning only — never at runtime).

1. Stage 0 — dry run on the Mac, no hardware

Get the whole stack green with everything simulated. Any failure after this point is unambiguously a hardware or network problem, not a software one.

```
cd ~/Desktop/Home_Automator/home_automation/server
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

Three dependencies only: `flask`, `pyserial`, `requests`.

Create the config file at the **repo root** (not in `server/`):

```
cd ~/Desktop/Home_Automator/home_automation
cp .env.example .env
```

For now, set just one line in `.env`:

```
MOCK_HARDWARE=1
```

Run it:

```
cd server
.venv/bin/python run.py
```

Open http://localhost:8000.

What you should see in the log:

```
Database ready at .../data/home.db
Running with MOCK_HARDWARE=1 — all hardware is simulated
MOCK_HARDWARE=1: generating fake sensor data (no serial port)
Polling 2 wifi plug(s) every 10.0s
Auto-lighting job covering 2 WLED zone(s), checking every 30.0s
```

What you should see in the browser: the Board view with four room-condition widgets slowly drifting, two plug pairs with wobbling wattage, two lighting cards, and a motion section. The header VIEW switch should reach Planner and Health.

Run the test suite too — 111 tests, all under mock hardware:

```
cd server && .venv/bin/python -m unittest discover -s tests
```

> **Note on `.env` precedence.** The loader uses `os.environ.setdefault`, so a real shell variable **wins** over the file. `MOCK_HARDWARE=1 python run.py` overrides whatever `.env` says — handy for a one-off, and a trap if you forget you exported it.

2. Stage 1 — wired lane: Arduino Due + sensors

2.1 The one rule that will kill your board

The Arduino Due runs at 3.3 V and its I/O pins are not 5 V tolerant. Putting 5 V on an input pin can permanently damage the SAM3X. Every I2C breakout gets its power from the **3.3 V pin**, which also keeps the shared I2C bus pulled up to 3.3 V.

There is exactly one exception: the **HC-SR501 PIR** needs 5 V for its onboard regulator, but its output signal swings to 3.3 V natively, so it is safe on a Due input. Feed it 5 V, read its output on D2, and do not "helpfully" move it to 3.3 V — it will simply stop working.

Work with the USB cable unplugged. Plug in only after you have re-checked every wire.

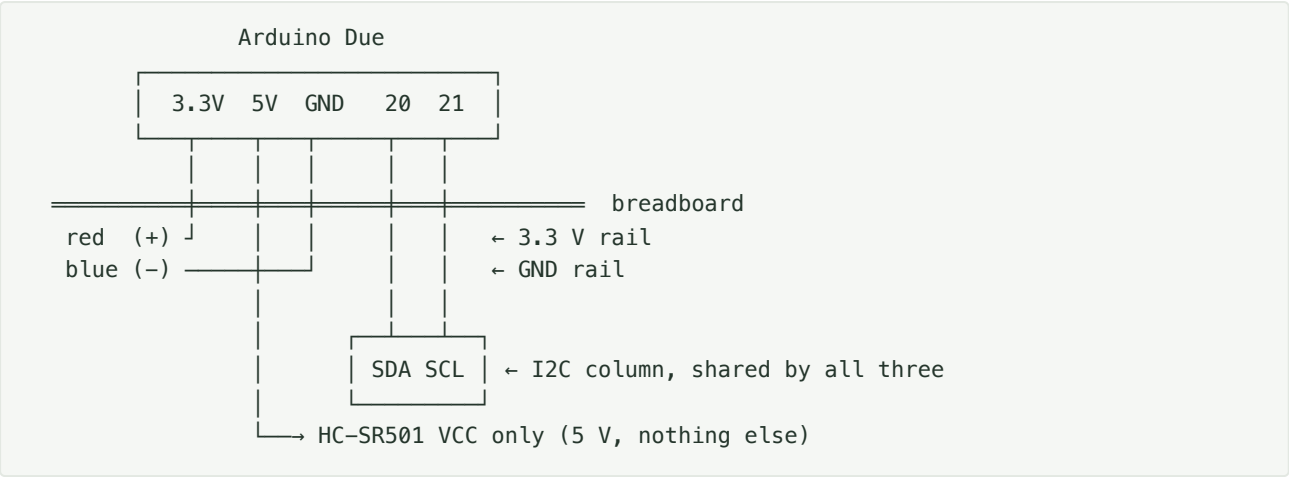
2.2 Pinout

DEVICE	ITS PIN	→ DUE PIN
BME280	VIN / VCC	**3.3V**
	GND	GND
	SCL	21
	SDA	20
BH1750	VCC	**3.3V**
	GND	GND
	SCL	21
	SDA	20
SCD40	VIN	**3.3V**
	GND	GND
	SCL	21
	SDA	20
HC-SR501	VCC	**5V** ← the exception
	GND	GND
	OUT	**D2**

Reserved for later (stubs already exist in the firmware, nothing to wire now): D7 relay IN1, D9 MOSFET gate (PWM), D6 addressable-LED data.

2.3 Build order

Build the power rails first, then hang sensors off them. This keeps you from running eight individual wires to the Due's header and miscounting one.



1. Jumper Due **3.3V** → breadboard red rail. Jumper Due **GND** → blue rail.
2. Pick two spare breadboard columns as the **SDA column** and **SCL column**. Jumper Due pin **20** → SDA column, pin **21** → SCL column.
3. Seat BME280, BH1750, and SCD40 on the board. For each: VCC → red rail, GND → blue rail, SDA → SDA column, SCL → SCL column. All three sit in parallel on the same two signal lines — that is what an I2C bus is.
4. HC-SR501: VCC → **directly to the Due's 5 V pin** (not the red rail — the red rail is 3.3 V), GND → blue rail, OUT → Due D2.
5. Re-check that no I2C breakout is touching 5 V. Then plug in the USB cable.

Current budget: the Due's 3.3 V rail comfortably supplies these — BME280 ~1 mA, BH1750 ~0.2 mA, SCD40 peaks around 205 mA during a measurement. If the board browns out or resets when the SCD40 samples, power the Due from its DC barrel jack (7–12 V) instead of relying on the laptop's USB port.

2.4 Libraries and flashing

In the Arduino IDE, install the board core: **Tools → Board → Boards Manager → "Arduino SAM Boards (32-bits ARM Cortex-M3)"**.

Then **Tools → Manage Libraries** and install:

LIBRARY	FOR
Adafruit BME280 Library	temperature / humidity
Adafruit Unified Sensor	dependency of the above
BH1750 (Christopher Laws)	ambient light
SparkFun SCD4x Arduino Library	CO2

`Wire` is built in. The PIR needs no library.

Open `firmware/hub_node/hub_node.ino`. Select **Board: Arduino Due (Programming Port)** and the port under **Tools → Port**. Upload.

> **Programming port, not native port.** The Due has two micro-USB sockets. Use the one **nearer the DC barrel jack** — that is `Serial` in the sketch and the port the host reads. The native port will look like it works and then not.

2.5 Verify on the serial monitor

Open the Arduino serial monitor at **115200 baud**. Within a few seconds:

```
# hub_node boot
# BME280: ok
# BH1750: ok
# SCD40:  ok
TEMP:21.4
HUM:47.2
LUX:312
CO2:612
MOTION:0
```

Lines starting with `#` are firmware logs; the host ignores them. Readings arrive every 5 s, and `MOTION` is additionally sent the instant the PIR changes.

A `NOT FOUND` line means that sensor did not answer on the I2C bus. It is not fatal — the firmware just skips it and that metric never appears — but fix it now:

- Re-check VCC/GND/SDA/SCL on that specific board.
- Run an I2C scan (§2.6) to see what the bus actually reports.
- A BME280 clone may be at 0x76 while an Adafruit board is at 0x77; the firmware already tries both, so an address mismatch is not the cause here.

2.6 I2C scanner

The single most useful diagnostic. Upload this temporarily, then re-flash `hub_node.ino` afterwards:

```
#include <Wire.h>
void setup() {
  Serial.begin(115200);
  while (!Serial);
  Wire.begin();
  Serial.println("Scanning I2C...");
  for (uint8_t a = 1; a < 127; a++) {
    Wire.beginTransmission(a);
    if (Wire.endTransmission() == 0) {
      Serial.print("Found 0x"); Serial.println(a, HEX);
    }
  }
  Serial.println("Done.");
}
void loop() {}
```

You should see `0x23` (BH1750), `0x62` (SCD40), and `0x76` or `0x77` (BME280).

- **Nothing found at all** → SDA/SCL swapped, or no power to the breakouts, or the wires are in the wrong breadboard row. Check continuity before suspecting the chips.

- **Some addresses found, one missing** → that one board's wiring, specifically.
- **Addresses appear and disappear between scans** → bus integrity. Shorten the jumper wires. If the bus is still flaky with all three boards attached, total pull-up strength is the usual culprit: the Due carries pull-ups on pins 20/21 and each breakout adds its own in parallel, so several boards together can over-pull the bus. Removing the pull-up resistors from all but one breakout fixes it.

2.7 Sensor-specific quirks

HC-SR501 (PIR) — give it **~60 seconds** after power-on before you trust `MOTION`; it self-calibrates and will emit spurious triggers during that window. It has two trimpots: sensitivity (range) and time-delay (how long the output stays HIGH after a trigger). Set the time delay near its minimum — the hub timestamps events itself and a long hold just smears them together. Some boards also have an H/L jumper; choose **repeat trigger (H)** so continuous presence keeps the output HIGH.

SCD40 (CO2) — needs a minute or two after power-up before readings settle, and it self-calibrates on the assumption that it periodically sees fresh outdoor air (≈ 420 ppm). If installed somewhere that never gets aired out, readings drift. If values look wrong after a week, run a forced recalibration with the sensor outdoors. Values above ~1000 ppm mean ventilate — that is the default alert threshold.

BME280 — self-heats slightly and will read a few tenths of a degree above ambient if crammed against other components or in a closed enclosure. Give it airflow and keep it away from the Due's regulator.

BH1750 — reports true lux, so it is directly meaningful for the auto-lighting threshold. Mount it where it sees the room's *ambient* level, not pointing at a lamp or a window, or the auto-lighting job will oscillate.

3. Stage 2 — connect the Due to the Mac

Find the serial port:

```
ls /dev/tty.usb*
```

You will get something like `/dev/tty.usbmodem14101`. Put it in `.env`, and turn mock mode off:

```
SERIAL_PORT=/dev/tty.usbmodem14101
SERIAL_BAUD=115200
MOCK_HARDWARE=0
```

Close the Arduino serial monitor. Only one process can hold the port; if the IDE has it, the hub will log `SERIAL UNAVAILABLE` forever.

Restart the server. You want:

```
Serial connected on /dev/tty.usbmodem14101 @ 115200 baud
```

Sensor rows should start landing immediately. Confirm from the API:

```
curl -s localhost:8000/api/sensors/latest | python3 -m json.tool
```

You should see `temp`, `hum`, `lux`, `co2`, `motion` with fresh timestamps. Wave at the PIR and re-check that `motion` flips to 1.

Failures here are loud and non-fatal by design: the reader logs `SERIAL UNAVAILABLE` or `SERIAL DROPPED` and retries every 5 s. Unplugging the USB cable and plugging it back in should recover on its own without restarting the server — worth testing once now, because that is exactly what will happen at 3 a.m. six months from now.

4. Stage 3 — WiFi lane: myStrom plugs

Runtime control is **local REST only** — no cloud, no account. The app is touched once, to get the plug onto WiFi.

4.1 Provision

1. Plug the switch into a socket. The LED blinks **red, short** = access-point mode.
2. Easiest path: the **myStrom app** → add device → follow the WiFi flow.
3. App-free alternative: join the open WiFi network named `my-xxxxxx` (no password) and open **http://192.168.254.1** in a browser to enter your WiFi credentials.

4.2 Give it a fixed address

In your router's DHCP settings, add a **static reservation** for the plug's MAC. Do not skip this — the hub stores the IP, and a lease change silently breaks the device until you notice `PLUG UNREACHABLE` in the logs.

Suggested: `192.168.0.51` for Plug 1, `192.168.0.52` for Plug 2 (matching the defaults in `.env.example`).

4.3 Verify before touching the hub

```
curl -s http://192.168.0.51/report
# {"power":0,"relay":false}

curl -s "http://192.168.0.51/relay?state=1"   # on
curl -s "http://192.168.0.51/relay?state=0"   # off
curl -s http://192.168.0.51/toggle            # flip
curl -s http://192.168.0.51/temp              # internal temperature
```

If `/report` answers, the hub will work. If it does not, no amount of configuration on the hub side will help.

> The myStrom local API has **no authentication**. Anyone on your LAN can switch the plug. That is inherent to the device, and the reason the hub is reached over Tailscale rather than a port forward.

4.4 Tell the hub

Put the addresses in `.env`:

```
MYSTROM_PLUG_IP=192.168.0.51
MYSTROM_PLUG2_IP=192.168.0.52
MYSTROM_POLL_INTERVAL=10
```

> **Important gotcha.** Device rows are seeded **insert-if-missing**. Editing an IP in `.env` does *not* update a row that already exists — which it does, because stage 0 already created it from the placeholder defaults. You must either update the row or delete the database and let it reseed:
>
> ```
> sqlite3 data/home.db \
>   "UPDATE devices SET ip='192.168.0.51' WHERE name='Plug 1';
>    UPDATE devices SET ip='192.168.0.52' WHERE name='Plug 2';"
> ```
>
> Restart the server afterwards — the poller builds its client list at startup.

Confirm: `PLUG UNREACHABLE` stops appearing, and the plug's power widget on the dashboard shows a real wattage. Toggle it from the dashboard and watch a lamp respond.

4.5 Naming, rooms, and locking

The seeds are `Plug 1` (Living Room) and `Plug 2` (Unassigned). Rename to match reality:

```
sqlite3 data/home.db "UPDATE devices SET name='Desk Lamp', room='Study' WHERE name='Plug 2';"
```

Scene targets keyed by device *name* reference these strings — if you rename a plug that a scene mentions by name, update the scene's `states` JSON too. Scenes using the `all_plugs` group key are unaffected.

Lock any plug that must never be switched off by a scene or a stray tap — a fridge, a router, a NAS:

```
sqlite3 data/home.db "UPDATE devices SET locked=1 WHERE name='Fridge';"
```

Locked plugs are skipped by scene activation and reported per-device in the response.

5. Stage 4 — WiFi lane: Shelly bulbs

5.1 Physical

Screw each bulb into its E27 fixture. The bulb is the entire device — there is no controller, no strip, no soldering. **The fixture's wall switch must stay ON permanently**; cutting mains makes the bulb unreachable, and the hub will just log it as offline. If the fixture has an accessible switch, tape it or use a fixture without one.

5.2 Provision

1. Power the bulb. On first boot it opens its own access point.
2. Easiest path: the **Shelly Smart Control** app → add device.
3. App-free alternative: join the bulb's AP and open **http://192.168.33.1** → **Settings → Wi-Fi** → enter your network and password.
4. **Factory reset** if you mistype the credentials: toggle mains power off/on five times in a row; the bulb returns to AP mode.

Give each bulb a **static DHCP reservation**, same as the plugs. Suggested `192.168.0.61` (Cupboard) and `192.168.0.62` (Table).

Cloud can be left disabled — nothing at runtime needs it.

5.3 Verify locally

The Multicolor Bulb E27 Gen3 exposes an `rgbcct:0` component over Gen2+ JSON-RPC. Note this is `RGBCCT.*`, **not** the `Light.*` methods used by simpler Shelly dimmers.

```
# identity
curl -s http://192.168.0.61/rpc/Shelly.GetDeviceInfo

# state
curl -s "http://192.168.0.61/rpc/RGBCCT.GetStatus?id=0"

# on, half brightness, warm orange
curl -s "http://192.168.0.61/rpc/RGBCCT.Set?id=0&on=true&brightness=50&rgb=[255,176,102]"

# off
curl -s "http://192.168.0.61/rpc/RGBCCT.Set?id=0&on=false"
```

Or as JSON-RPC POST, which is the form the hub client should use:

```
curl -s -X POST http://192.168.0.61/rpc \
  -H 'Content-Type: application/json' \
  -d '{"id":1,"method":"RGBCCT.Set","params":{"id":0,"on":true,"brightness":50}}'
```

Key parameters of `RGBCCT.Set`:

PARAM	RANGE	NOTE
`id`	—	always `0` on a single-light bulb
`on`	bool	

PARAM	RANGE	NOTE
`brightness`	**1–100**	**percent**, not 0–255
`rgb`	`[0–255, 0–255, 0–255]`	
`ct`	Kelvin (~2700–6500)	color-temperature mode
`mode`	`"rgb"` | `"cct"`	
`transition_duration`	seconds	

`RGBCCT.GetStatus` returns `output` (bool), `brightness`, `rgb`, `ct`, `mode`.

If you set a device password in the Shelly web UI, RPC then requires digest auth — leave it unset on a Tailscale-only LAN, or the client gains a dependency it does not currently have.

5.4 The Shelly migration (still to do)

Once both bulbs answer `curl`, the remaining work is on the hub side. The codebase targets WLED end to end; nothing will drive a Shelly bulb until this is done.

The two substantive design decisions:

- **Brightness scale.** The hub is 0–255 everywhere (`LIGHTING_AUTO_BRIGHTNESS=180`, the dashboard slider `min="0" max="255"`, `api.py` validation, scene seeds). Shelly is 1–100 %. Convert **inside the client** — `pct = max(1, round(b / 255 * 100))` outbound, `b255 = round(pct * 255 / 100)` inbound — and every other layer stays unchanged. Do not renumber the hub.
- **Effects.** WLED has numbered effects (`fx`); the Shelly bulb has no equivalent. Drop the effect control from the lighting card, or repurpose it as the `rgb` / `cct` mode toggle. Note `CLAUDE.md` contradicts itself here — the bulb-integration section describes the client as `on` / `brightness` / `color`, while the API list still carries `effect` on `POST /api/devices/:id/state`. Settle it in the same commit.

Files to change:

FILE	CHANGE
`server/app/shelly_bulb.py`	**new** — replaces `wled.py`. `state()` / `set_state()` over `RGBCCT.*`, plus `MockShellyBulb` for `MOCK_HARDWARE=1` and a `BulbError`
`server/app/wled.py`	delete once the above lands
`server/app/config.py`	`WLED_CUPBOARD_IP` / `WLED_TABLE_IP` → `SHELLY_CUPBOARD_IP` / `SHELLY_TABLE_IP`
`server/app/db.py`	`WLED_SEEDS` → `BULB_SEEDS`; type `'wled_zone'` → `'bulb_zone'`; add a one-time migration `UPDATE devices SET type='bulb_zone' WHERE type='wled_zone'`
`server/app/lighting.py`	import, type filter, `WledError` → `BulbError`
`server/app/api.py`	`_wled_state_or_none`, `_attach_device_state`, `device_state`, `device_mode` — type checks and error strings
`server/app/scenes.py`	`_apply_zone` (drop the `effect` argument)
`dashboard/app.js`	type strings at lines ~341 and ~607; effect control
`dashboard/index.html`	lighting card markup if the effect control goes

FILE	CHANGE
`.env.example`	rename the two IP vars
`server/tests/test_scenes.py`, `test_planner.py`	`wled_zone` references
`docs/api.md`, `docs/physical-setup.md`, `server/README.md`	wording

Keep `MOCK_HARDWARE=1` working throughout — the test suite depends on it, and it is how you verify the migration without unscrewing a bulb.

After the migration, set a zone's mode to `auto` from its lighting card (or `POST /api/devices/:id/mode {"mode":"auto"}`) to hand its brightness to the lux-driven job. Tune with `LIGHTING_LUX_THRESHOLD` (default 50 lux) and `LIGHTING_AUTO_BRIGHTNESS` (default 180/255 ≈ 70 %).

6. Stage 5 — full verification on the Mac

Work down this list before touching the Pi. Everything except the two starred rows should pass.

#	CHECK	HOW	EXPECTED
1	Serial up	server log	`Serial connected on /dev/tty.usbmodem…`
2	All five metrics	`curl -s localhost:8000/api/sensors/latest`	`temp`, `hum`, `lux`, `co2`, `motion`, fresh timestamps
3	Motion is live	wave at the PIR	Motion widget flips within a second
4	History accumulates	leave it an hour, open a widget	Chart has a real curve
5	Plug 1 reachable	dashboard toggle	Lamp switches; wattage changes
6	Plug 2 reachable	dashboard toggle	As above
7	Power history	expand a plug widget	24 h series + kWh estimate
8	Thresholds	gear icon, narrow the temp band	Widget flags immediately
9	Scenes	header MODE → Away	Unlocked plugs off; locked ones reported skipped
10	Locked plug honored	lock one, activate Away	It stays on
11	Wake timer	Sleeping with wake time 2 min out	Switches to Day on its own; summary card appears
12	Serial recovery	unplug the USB, wait, replug	`SERIAL DROPPED` then reconnect, no restart
13	Planner	add an event and a task	Persist across a restart
14	Tests	`.venv/bin/python -m unittest discover -s tests`	111 pass
15 ★	Bulb reachable	`curl .../rpc/RGBCCT.GetStatus?id=0`	JSON — but the **hub cannot drive it yet** (§5.4)
16 ★	Auto-lighting	—	Blocked on the same migration

Leave it running for a day or two before migrating. Sensor drift, a plug dropping off WiFi, or a scene misfiring are all much easier to diagnose on the machine you are sitting at.

7. Stage 6 — the Raspberry Pi 4

Same code, no modifications — the server deliberately contains nothing Pi-specific. What changes is the OS, the serial device path, and that it now runs as a service.

7.1 Image the SD card

The Pi boots from an ordinary microSD card. Nothing in the hub is tied to the storage medium — `DB_PATH` is an env var and the server contains nothing machine-specific — so this is revisitable later without touching code; §7.6 is the entire migration.

Size is not the constraint, endurance is. The hub writes roughly 104 000 small rows a day (five sensor metrics every 5 s, two plug samples every 10 s) — about 8 MB/day, so ~3 GB a year with indexes, plus ~0.5 GB a year once health data flows. A 32 GB card is years of headroom. What kills Pi cards is not capacity but sustained small writes, so spend on the endurance rating rather than on more space you will not use.

> **Nothing ever deletes a reading.** There is no retention policy, no downsampling and no `VACUUM` anywhere in the codebase, so the tables grow linearly forever. At ~3 GB a year that is fine for years — but it will not plateau on its own, so it is worth knowing before the card is 80 % full.

1. Install **Raspberry Pi Imager** on the Mac.
2. Insert the card. Choose **Raspberry Pi OS Lite (64-bit)** — no desktop, this is a headless server and the RAM budget matters.
3. In Imager's settings (gear icon), pre-configure: hostname (e.g. `hub`), your username, **enable SSH** with your public key, WiFi credentials, and **locale + timezone**.
4. Write to the card.

> **Timezone is not cosmetic here.** Scene wake times, the planner's local `"YYYY-MM-DDTHH:MM"` parsing, and the health module's noon-to-noon night anchoring all use local wall-clock time. A Pi left on UTC will put readings on the wrong night.

7.2 First boot

```
ssh <user>@hub.local
sudo apt update && sudo apt full-upgrade -y
sudo apt install -y git python3-venv sqlite3
```

Confirm the timezone and that the card came up as the root filesystem:

```
timedatectl        # check "Time zone"
df -h /            # ~29 GB usable on a 32 GB card
```

Then cut the background writes the card sees. These two are most of the difference between a card that lasts years and one that does not:

```
sudo sed -i 's/^#\?SystemMaxUse=.*/SystemMaxUse=50M/' /etc/systemd/journald.conf
sudo systemctl restart systemd-journald
sudo systemctl disable --now dphys-swapfile
```

The journal is otherwise unbounded and this service logs continuously; swap on flash is pure wear for a workload that never comes near 4 GB.

7.3 Get the code

```
git clone <your-remote> ~/home_automation
cd ~/home_automation/server
python3 -m venv .venv
.venv/bin/pip install -r requirements.txt
```

All three dependencies are pure-Python wheels — no compilation, no `build-essential`.

7.4 Serial port on Linux

The Due's programming port enumerates as `/dev/ttyACM0`, but that number can change if anything else USB-serial is ever attached. Use the stable by-id path instead:

```
ls -l /dev/serial/by-id/
# usb-Arduino__www.arduino.cc__Arduino_Due_Prog._Port_<serial>-if00
```

Put that full path in `SERIAL_PORT`.

Your user needs permission on the port:

```
sudo usermod -aG dialout $USER
```

Log out and back in for the group to take effect.

> **ModemManager will steal your Arduino.** On a stock Debian-based image, ModemManager probes new `ttyACM` devices and can hold the port for seconds at a time, giving you intermittent `SERIAL UNAVAILABLE`. On a dedicated hub, remove it:
>
> ```
> sudo apt purge -y modemmanager
> ```

7.5 Configure

```
cp ~/home_automation/.env.example ~/home_automation/.env
nano ~/home_automation/.env
```

```
SERIAL_PORT=/dev/serial/by-id/usb-Arduino__www.arduino.cc__Arduino_Due_Prog._Port_XXXX-if00
SERIAL_BAUD=115200

DB_PATH=/home/<user>/home_automation/data/home.db

MYSTROM_PLUG_IP=192.168.0.51
MYSTROM_PLUG2_IP=192.168.0.52
MYSTROM_POLL_INTERVAL=10

# rename these two once §5.4 is done
WLED_CUPBOARD_IP=192.168.0.61
WLED_TABLE_IP=192.168.0.62

LIGHTING_POLL_INTERVAL=30
LIGHTING_LUX_THRESHOLD=50
LIGHTING_AUTO_BRIGHTNESS=180

HOST=0.0.0.0
PORT=8000
MOCK_HARDWARE=0
```

Everything lives on the one card — OS, venv, database, backups — so `DB_PATH` is just a path under your home directory. There is no separate mount to point at.

Run it in the foreground once and watch the log before making it a service:

```
cd ~/home_automation/server && .venv/bin/python run.py
```

7.6 Bring the Mac's data over (optional)

If you want to keep the history recorded during stages 1–5, copy the database **with SQLite's backup command**, not `cp` — the DB runs in WAL mode and a plain copy of a live file can be inconsistent.

On the Mac, with the server stopped:

```
sqlite3 data/home.db ".backup /tmp/home.db"
scp /tmp/home.db <user>@hub.local:~/home_automation/data/home.db
```

Device IPs come along with the rows. Re-check them if anything changed.

Starting fresh instead is perfectly fine — the tables reseed from `.env` on first run.

7.7 Run it as a service

```
sudo nano /etc/systemd/system/homehub.service
```

```
[Unit]
Description=Smart Home Hub
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=<user>
WorkingDirectory=/home/<user>/home_automation/server
ExecStart=/home/<user>/home_automation/server/.venv/bin/python run.py
Restart=always
RestartSec=5
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
```

`WorkingDirectory` must be `server/` — `run.py` imports `from app import ...` and relies on its own directory being on `sys.path`. The `.env` file is found at the repo root regardless, since `config.py` resolves it relative to itself.

```
sudo systemctl daemon-reload
sudo systemctl enable --now homehub
systemctl status homehub
journalctl -u homehub -f          # live log
```

Reboot and confirm it comes back on its own. That is the actual acceptance test for this stage:

```
sudo reboot
# then, after it comes up:
systemctl status homehub
curl -s localhost:8000/api/sensors/latest
```

7.8 Tailscale

```
curl -fsSL https://tailscale.com/install.sh | sh
sudo tailscale up
```

The dashboard is then at `http://hub:8000` from any device on your tailnet.

Do **not** port-forward 8000. The app has no authentication by design — Tailscale is the access control, as decided in `CLAUDE.md`.

7.9 Backups — not optional on a card

The database is the only irreplaceable thing on the box, and SD cards fail at exactly this workload: sustained small writes, ~17 000 commits a day, indefinitely. They usually go without warning. A backup sitting on the same card is worth nothing when that happens, which is why the second cron line below matters as much as the first.

`.backup`, never `cp` — the DB runs in WAL mode and a plain copy of a live file can be inconsistent.

```
mkdir -p ~/backups
ssh-keygen -t ed25519          # if the Pi has no key yet
ssh-copy-id <you>@<mac>        # so the nightly copy runs unattended
crontab -e
```

```
15 4 * * * /usr/bin/sqlite3 /home/<user>/home_automation/data/home.db ".backup
/home/<user>/backups/home-$(date +\%u).db"
30 4 * * * /usr/bin/scp -q /home/<user>/backups/home-$(date +\%u).db <you>@<mac>:~/hub-
backups/
```

The first keeps a rolling week on the card (`%u` is the weekday number); the second puts last night's copy somewhere that survives the card dying. The Mac needs **Remote Login** on (System Settings → General → Sharing) and needs to be awake at 04:30 — if it usually is not, move the second line to a time you are actually at the machine.

> **Check that it is arriving.** A silently failing `scp` — asleep Mac, changed host key, full disk — looks exactly like a working backup from the Pi's side. Look in `~/hub-backups/` on the Mac now and then, and confirm the timestamps move.

7.10 Health data ingest

The Health view is fed by the **Health Auto Export** iOS app POSTing Apple Health JSON to `POST /api/health/ingest`. Point it at your Tailscale address:

```
http://hub:8000/api/health/ingest
```

Disable **"aggregate sleep data"** in the app — the ingest endpoint needs per-stage segments with `startDate` / `endDate`, and rejects aggregated payloads whole with an explanatory error. See `docs/api.md` for the expected payload shape, including RR intervals (not a stock export metric).

Clear the seeded demo nights before real data flows — the Health view currently holds ~30 artificial nights, which would otherwise pollute your rolling baselines:

```
cd ~/home_automation/server && .venv/bin/python tools/seed_health.py --clear
```

7.11 Final placement

- The PIR needs line of sight to where you actually walk; it sees heat *movement across* its field, not toward it.
- The BH1750 wants a view of ambient room light — not aimed at a lamp, not at a window, or auto-lighting will chase itself.
- The SCD40 needs air circulation, away from where you breathe directly on it (a person at close range will spike it by hundreds of ppm).
- Keep the Due's USB run short and use a decent cable. Long, thin USB cables are a genuinely common cause of intermittent serial drops.
- The Pi is fanless and passive — leave clearance around it and do not box it in.

8. Day 2 — making changes

Once the Pi is live it holds the only copy of your real data and runs unattended. Develop on the Mac against synthetic data, deploy to the Pi through git.

Two `.gitignore` rules are what make this safe:

```
.env        # per-machine config never travels
*.db        # the Pi's real data can never be clobbered by a pull
```

A `git pull` on the Pi therefore touches code only — never your configuration, never your measurements.

> `*.db` is also what keeps your Apple Health data out of GitHub. Never `git add -f` a database.

8.1 The loop

On the **Mac** (`.env` has `MOCK_HARDWARE=1`):

```
cd server && .venv/bin/python run.py        # iterate at localhost:8000
.venv/bin/python -m unittest discover -s tests
cd .. && git add -A && git commit -m "..." && git push
```

On the **Pi**:

```
ssh <user>@hub
cd ~/home_automation
git pull
sudo systemctl restart homehub              # backend changes only
journalctl -u homehub -f                    # watch it come up
```

Changes confined to `dashboard/` need only the `git pull` and a hard refresh in the browser — no restart. Flask serves that directory as static files straight off disk, and sensor ingestion runs in background threads that never touch the frontend, so there is no gap in the data.

8.2 `.env` does not travel — and missing vars fail silently

Every setting in `config.py` has a default. A variable you added on the Mac but forgot on the Pi will **not** raise an error; it quietly falls back, and the feature misbehaves in a way that looks like a bug in the code.

Update `.env.example` in the same commit as the code that reads the new variable, then on the Pi check what is missing:

```
comm -23 <(grep -oE '^[A-Z_]+' .env.example | sort) <(grep -oE '^[A-Z_]+' .env | sort)
```

Anything listed is present in the example but absent from your live config.

8.3 Schema changes deploy themselves — one way only

`db.init_db()`, `planner.init_db()`, and `health.init_db()` all run at startup with `CREATE TABLE IF NOT EXISTS` plus `ALTER TABLE` column additions, so a schema change applies on the next restart with no manual migration step.

Convenient, but there is no down-migration, and `init_db()` does more than add columns — it also rewrites rows (see the `_LEGACY_SCENE_STATES` handling). **Back up before any deploy that touches DDL:**

```
sqlite3 data/home.db ".backup ~/backups/pre-deploy-$(date +%F).db"
```

8.4 Never edit files directly on the Pi

Any local modification makes the next `git pull` conflict, and you will find out at the worst possible moment. If you genuinely must hotfix on the Pi, commit and push **from** the Pi so the Mac can pull it back.

VS Code Remote-SSH is worth having for problems that only reproduce on the Pi — serial timing, a plug dropping off WiFi, an SCD40 misbehaving. Use it to *diagnose*, then fix on the Mac and deploy normally. The failure mode to avoid is letting uncommitted edits accumulate on the hub.

8.5 Mock mode does not exercise the hardware seams

`MOCK_HARDWARE=1` replaces `serial_reader`, `mystrom`, and the bulb client wholesale. Changes to those three files — or to anything depending on their real timing and error behavior — are genuinely untested until they reach the Pi. Deploy those while watching `journalctl -u homehub -f`, not fire-and-forget.

8.6 Do not point the Mac at the real devices while the Pi is running

If you set `MOCK_HARDWARE=0` on the Mac to test the WiFi lane for real, you now have **two hosts polling and commanding the same devices**. `lighting.py` pushes brightness to every `auto` zone every 30 s from both machines, so they will fight over the zones; scene activation from either moves real hardware. Stop the hub first:

```
sudo systemctl stop homehub
```

Note that `MOCK_HARDWARE` is all-or-nothing — one flag gates all three lanes, so there is no "fake the serial port but talk to the real plugs" mode today. Splitting it into `MOCK_SERIAL` / `MOCK_WIFI` is a small change and would be genuinely useful while building the Shelly client (§5.4), since a mock you wrote yourself is the one thing that cannot validate your own protocol assumptions.

8.7 Rollback

Tag deploys you trust. Getting back is two commands:

```
git checkout <sha> && sudo systemctl restart homehub
```

If the bad deploy included a schema migration, restore the backup from §8.3 as well. Return to the tip with `git checkout main`.

9. Troubleshooting

SYMPTOM	LIKELY CAUSE	FIX
`SERIAL UNAVAILABLE: cannot open …`	Wrong path, Arduino unplugged, or another program holds the port	Check `ls /dev/tty.usb*` (Mac) or `/dev/serial/by-id/` (Pi); close the Arduino serial monitor; purge ModemManager on the Pi
`SERIAL DROPPED`, repeatedly	Cable or power	Shorter/better USB cable; power the Due from its DC jack
Serial connects, no readings	Sketch not running, or wrong USB socket	Use the **programming port**; re-flash; check the monitor at 115200
`# BME280: NOT FOUND` at boot	Wiring or bus	Run the I2C scanner (§2.6)
One metric missing, others fine	That sensor failed to init	Same — scan the bus, re-check that board's four wires
`Unrecognized serial line (protocol drift?)`	Firmware and host parser disagree	Keep `hub_node.ino`, `serial_reader.py`, and `docs/serial-protocol.md` in sync — change all three in one commit
Motion always 1	PIR still warming up, or trimpots at max	Wait 60 s; reduce sensitivity and time-delay
CO2 stuck or implausible	SCD40 not settled, or ASC never sees fresh air	Wait a few minutes; force recalibration outdoors
`PLUG UNREACHABLE`	Wrong IP, or DHCP moved it	`curl http://<ip>/report`; add a static reservation; **update the DB row**, not just `.env`
Changed an IP in `.env`, nothing happened	Seeds are insert-if-missing	`UPDATE devices SET ip=…` and restart (§4.4)
Plug ignores a scene	It is locked	By design — `UPDATE devices SET locked=0` if you meant otherwise
`WLED ZONE UNREACHABLE`	Expected until §5.4 is done	The seeded zone IPs are placeholders
Bulb offline	Its fixture's wall switch is off	The bulb needs permanent mains
Auto-lighting oscillates	BH1750 sees the light it controls	Move the sensor; widen `LIGHTING_LUX_THRESHOLD`
Auto-lighting does nothing	A non-Day scene is active, or the zone is `manual`	By design — the card reads "Auto paused"; switch to Day
Wake timer did not fire	Backend was down past the time	It fires immediately at startup instead; check that the service is enabled
Health ingest rejected	Aggregated sleep data	Turn off "aggregate sleep data" in Health Auto Export
Dashboard blank / stale	Server down	`systemctl status homehub`; `journalctl -u homehub -n 50`
Service will not start	`WorkingDirectory` or venv path wrong	Both must be absolute and point into `server/`

SYMPTOM	LIKELY CAUSE	FIX
Filesystem goes read-only, or SQLite reports `disk I/O error`	The card is failing or its filesystem is corrupt	Do not fight it — image a fresh card (§7.1) and restore the newest backup from the Mac (§7.9)
Card filling up	Nothing prunes the tables, ever	`du -h data/home.db`; growth is ~3 GB/year and linear (§7.1)

Appendix A — environment variables

Full list in `.env.example` and `server/app/config.py`. The ones that matter during bring-up:

VARIABLE	DEFAULT	NOTES
`SERIAL_PORT`	`/dev/tty.usbmodem14101`	Mac: `/dev/tty.usb*`. Pi: use `/dev/serial/by-id/…`
`SERIAL_BAUD`	`115200`	Must match the firmware
`DB_PATH`	`./data/home.db`	On the Pi, anywhere on the card — the default is fine
`MYSTROM_PLUG_IP` / `_PLUG2_IP`	`192.168.0.51` / `.52`	Seeds only — see §4.4
`MYSTROM_POLL_INTERVAL`	`10` (s)	Plug state + power sampling
`WLED_CUPBOARD_IP` / `WLED_TABLE_IP`	`192.168.0.61` / `.62`	Rename to `SHELLY_*` in §5.4
`LIGHTING_POLL_INTERVAL`	`30` (s)	Auto-lighting tick
`LIGHTING_LUX_THRESHOLD`	`50` (lux)	Below this counts as dark
`LIGHTING_AUTO_BRIGHTNESS`	`180` (0–255)	Applied when dark
`HOST` / `PORT`	`0.0.0.0` / `8000`	
`MOCK_HARDWARE`	`0`	`1` simulates serial, plugs, and zones

`HEALTH_*` variables (baseline windows, recovery/sleep weights, penalties) are tuning knobs, not setup — change them and run `POST /api/health/recompute`, which rebuilds scores from stored raw data without re-ingesting.

Appendix B — serial protocol card

115200 baud, line-based, `\n`-terminated. Full spec in `docs/serial-protocol.md`. **If you change it, change `firmware/hub_node/hub_node.ino` and `server/app/serial_reader.py` in the same commit.**

Arduino → host, every 5 s (`MOTION` also on change):

```
TEMP:21.4     HUM:47.2     LUX:312     CO2:612     MOTION:1
```

Host → Arduino (all currently stubs, wired later):

```
RELAY1:ON | RELAY1:OFF        D7
DIM1:0-255                    D9 PWM
MODE:<name> | COLOR:r,g,b     D6
```

Lines starting with `#` are firmware logs — ignored by the host parser.

Send one by hand:

```
curl -s -X POST localhost:8000/api/arduino/command \
  -H 'Content-Type: application/json' -d '{"command":"RELAY1:ON"}'
```

Appendix C — useful one-liners

```
# Live log (Pi)
journalctl -u homehub -f

# Current state of everything
curl -s localhost:8000/api/sensors/latest | python3 -m json.tool
curl -s localhost:8000/api/devices       | python3 -m json.tool
curl -s localhost:8000/api/scenes/active | python3 -m json.tool

# Device registry
sqlite3 data/home.db "SELECT id,name,type,ip,room,mode,locked FROM devices;"

# Are readings actually landing?
sqlite3 data/home.db \
  "SELECT metric, COUNT(*), datetime(MAX(ts),'unixepoch','localtime') FROM readings GROUP BY
metric;"

# Database size
sqlite3 data/home.db "SELECT COUNT(*) FROM readings;"; du -h data/home.db

# Reset devices/scenes to seeds (destroys history — export first)
rm data/home.db && sudo systemctl restart homehub

# Safe backup of a live database
sqlite3 data/home.db ".backup /tmp/home-backup.db"
```

Sources

- myStrom WiFi Switch REST API
  https://mystrom.com/wp-content/uploads/REST_API_WSE.txt
  — `/report`, `/relay?state=`, `/toggle`, `/temp`
- myStrom WiFi Switch support
  https://mystrom.ch/support/wifi-switch/
  — AP mode, `192.168.254.1`
- Shelly RGBCCT Bulb Gen3
  https://shelly-api-docs.shelly.cloud/gen2/Devices/Gen3/ShellyRGBCCTBulbG3/
  — device components
- Shelly RGBCCT component
  https://shelly-api-docs.shelly.cloud/gen2/ComponentsAndServices/RGBCCT
  — `RGBCCT.Set` / `.GetStatus` parameters
- Adding a Shelly to WiFi via the web interface
  https://shelly.guide/add-a-shelly-to-your-wi-fi-through-web-interface/
  — AP mode, `192.168.33.1`